

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ”
ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №4

Специальность ПО-13

Выполнил:
Босак В.Ю.
студент группы
ПО-13
Проверил:
А.Д.Кулик
13.03.2026

Брест 2026

Цель работы: научиться работать с Github API, приобрести практические навыки написания программ для работы с REST API.

Ход работы

Общее задание: используя Github API, реализовать предложенное задание на языке Python. Выполнить визуализацию результатов, с использованием графика или отчета. Можно использовать как REST API (рекомендуется), так и GraphQL.

4. Анализ популярных технологий в открытых репозиториях GitHub

Условие:

Напишите Python-скрипт, который:

1. Запрашивает у пользователя ключевое слово или тему (например, "machine learning", "web development", "blockchain").
2. Использует GitHub API для поиска 100+ самых популярных репозиторий по этому ключевому слову.
3. Для каждого найденного репозитория собирает:

Язык программирования

Количество звезд

Количество форков

Количество открытых issues

Дату последнего обновления

4. Анализирует какие технологии чаще всего используются в данной области, строя рейтинг языков программирования.

5. Визуализирует результаты:

☐ Диаграмму популярных языков программирования

(matplotlib, seaborn, plotly)

График популярности репозиторий (по звёздам и активности)

График "старения" репозиторий (когда последний коммит)

Выполнение:

Код программы:

```
import requests
import matplotlib.pyplot as plt
from datetime import datetime, timezone

GITHUB_API = "https://api.github.com/search/repositories"
HEADERS = {"Accept": "application/vnd.github+json", "User-Agent": "Python-Script"}
```

```

def search_repos(topic, per_page=100):
    repos = []
    for page in range(1, 6):
        url =
f"{GITHUB_API}?q={topic}&sort=stars&order=desc&per_page=20&page={page}"
        response = requests.get(url, headers=HEADERS)
        if response.status_code != 200:
            print(f"Ошибка: {response.status_code}")
            break
        data = response.json()
        repos.extend(data.get("items", []))
        if len(repos) >= per_page:
            break
    return repos[:per_page]

def analyze(repos):
    lang_count = {}
    stars = []
    forks = []
    repo_names = []
    last_updated = []

    for repo in repos:
        lang = repo.get("language")
        if lang:
            lang_count[lang] = lang_count.get(lang, 0) + 1

        stars.append(repo.get("stargazers_count", 0))
        forks.append(repo.get("forks_count", 0))

        # Исправление: конвертируем строку в datetime с часовым поясом
        date_str = repo["updated_at"]
        date_obj = datetime.fromisoformat(date_str.replace("Z", "+00:00"))
        last_updated.append(date_obj)
        repo_names.append(repo["full_name"])

    total = sum(lang_count.values())
    lang_percent = {k: (v/total)*100 for k, v in lang_count.items()}

    max_stars_idx = stars.index(max(stars))
    top_repo = repo_names[max_stars_idx]
    top_stars = stars[max_stars_idx]

    avg_forks = sum(forks) / len(forks)

    # Исправление: используем datetime с часовым поясом

```

```

now = datetime.now(timezone.utc)
old_count = sum(1 for d in last_updated if (now - d).days > 365)
old_percent = (old_count / len(last_updated)) * 100

print(f"\nСамые популярные языки:")
for lang, percent in sorted(lang_percent.items(), key=lambda x: x[1],
reverse=True)[:5]:
    print(f"- {lang} ({percent:.1f}%)")

if len(lang_percent) > 5:
    other = 100 - sum(list(lang_percent.values())[:5])
    print(f"- Другие ({other:.1f}%)")

print(f"\nСамый звёздный репозиторий: \"{top_repo}\" ({top_stars}
звёзд)")
print(f"Среднее количество форков: {avg_forks:.1f}")
print(f"{old_percent:.0f}% репозитория не обновлялись больше года!")

return lang_percent, stars, repo_names, last_updated

def visualize(lang_percent, stars, repo_names, last_updated, topic):
    langs = list(lang_percent.keys())[:7]
    percents = list(lang_percent.values())[:7]

    plt.figure(figsize=(12, 4))

    plt.subplot(1, 3, 1)
    if sum(percents) > 0:
        plt.pie(percents, labels=langs, autopct="%1.0f%%")
    plt.title("Популярные языки")

    plt.subplot(1, 3, 2)
    top_stars = stars[:10]
    top_names = [n.split("/")[-1][:15] for n in repo_names[:10]]
    plt.barh(top_names[::-1], top_stars[::-1])
    plt.xlabel("Звёзды")
    plt.title("Топ-10 по звёздам")

    plt.subplot(1, 3, 3)
    now = datetime.now(timezone.utc)
    days_since = [(now - d).days for d in last_updated[:50]]
    plt.hist(days_since, bins=15, color="orange", edgecolor="black")
    plt.xlabel("Дней без обновлений")
    plt.ylabel("Кол-во репозитория")
    plt.title("Давность последнего обновления")

```

```

plt.tight_layout()
filename = f"{topic.replace(' ', '_')}_analysis.png"
plt.savefig(filename)
print(f"\nГрафики сохранены в \"{filename}\"")
plt.show()

def main():
    topic = input("Введите тему для анализа: ")
    print(f"Анализируем 100 популярных репозиторий по теме \"{topic}\"...")

    repos = search_repos(topic)
    if not repos:
        print("Репозитории не найдены")
        return

    lang_percent, stars, repo_names, last_updated = analyze(repos)
    visualize(lang_percent, stars, repo_names, last_updated, topic)

if __name__ == "__main__":
    main()

```

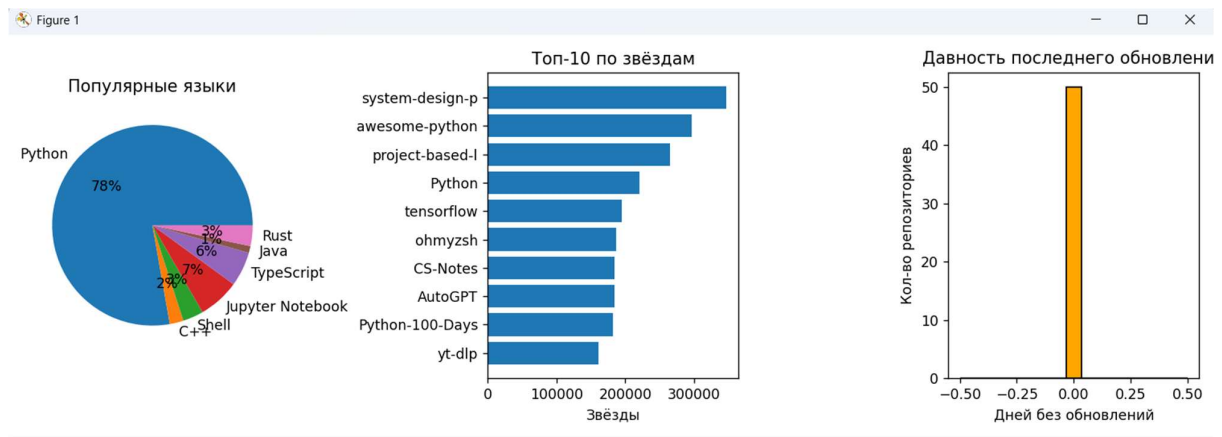
Рисунки с результатами работы программы

Введите тему для анализа: python
Анализируем 100 популярных репозиторий по теме "python"...

Самые популярные языки:

- Python (75.3%)
- Jupyter Notebook (6.5%)
- TypeScript (5.4%)
- Shell (3.2%)
- Rust (3.2%)
- Другие (7.5%)

Самый звёздный репозиторий: "donnemartin/system-design-primer" (347343 звёзд)
Среднее количество форков: 14920.6
0% репозиторий не обновлялись больше года!



Вывод: научился работать с Github API, приобрёл практические навыки написания программ для работы с REST API.